

Centro Universitário IESB
Ciência da Computação

Busca Sequencial – Fase I

Alunos: Lucas Pessoa Ranieri, Ana Luiza Gomes de Lima, Pedro Augusto Machado Cardoso

Brasília
2025

Centro Universitário IESB

Ciência da Computação

Relatório Técnico

Análise de Desempenho de Busca Sequencial

Relatório Técnico referente à Fase I do projeto de Análise de Desempenho de Algoritmos de Busca, desenvolvido na disciplina de Estruturas de Dados.

Alunos: Lucas Pessoa Ranieri, Ana Luiza Gomes de Lima, Pedro Augusto Cardoso Machado

Professor(a) orientador(a): José Reginaldo

Brasília
2025

Conteúdo

1	Resumo	1
2	Apresentação	2
2.1	Objetivo Geral	2
2.2	Estrutura de Dados	2
2.3	Dataset	2
2.4	Ambiente de Execução	3
3	Descrição de Atividades	4
3.1	Leitura do Arquivo CSV	4
3.2	Vetor Dinâmico	4
3.3	Implementação da Busca Sequencial	5
3.4	Medição de Tempo	5
3.5	Protocolo Experimental	6
4	Análise dos Resultados	8
4.1	Quantidade Total de Registros Carregados	8
4.2	Tempo Total das Execuções de Busca	8
4.3	Tempo Médio por Busca	8
4.4	Tabela de Resultados	9
4.5	Comportamento Observado	9
4.6	Relação Entre Tamanho do Vetor e Tempo de Busca	9
4.7	Limitações da Busca Sequencial	10
5	Trabalhos Futuros	11
	Bibliografia	12
	Anexo	13

1 Resumo

Este relatório técnico apresenta os resultados da Fase I do projeto de análise de desempenho de algoritmos de busca. O objetivo desta fase é estabelecer um *baseline* por meio da implementação e avaliação experimental da **busca sequencial** sobre um conjunto de **300.006 registros** de produtos armazenados em arquivo CSV. Foram realizadas 1.000 buscas consecutivas por rodada, repetidas 3 vezes para cada cenário de posição (início, meio, fim e elemento inexistente). Os resultados confirmaram o comportamento $\mathcal{O}(n)$ esperado para este algoritmo: o tempo médio por busca variou de **0,000 ms** (elemento no início) a **3,175 ms** (elemento no final), evidenciando a degradação linear conforme a posição do elemento no vetor. Os resultados servirão de referência comparativa para a Tabela Hash a ser implementada na Fase II.

2 Apresentação

2.1 Objetivo Geral

O projeto tem como objetivo desenvolver uma base experimental para análise de desempenho de algoritmos de busca, contemplando:

- Leitura e estruturação de dados reais a partir de arquivo CSV;
- Implementação da busca sequencial em linguagem C com vetor dinâmico;
- Medição sistemática do tempo de execução com múltiplas repetições;
- Construção de relatório técnico com análise dos resultados.

Esta fase estabelece o *baseline* que será utilizado como referência comparativa na implementação da Tabela Hash na Fase II.

2.2 Estrutura de Dados

O projeto utiliza duas estruturas principais definidas em `produto.h`. A estrutura `Produto` armazena cada registro do CSV, e a estrutura `VetorProdutos` gerencia o vetor dinâmico de registros:

```
1 typedef struct {  
2     int    id;  
3     char   nome[100];  
4     char   categoria[50];  
5     float  valor;  
6 } Produto;  
7  
8 typedef struct {  
9     Produto *dados;  
10    int      tamanho;  
11    int      capacidade;  
12 } VetorProdutos;
```

Listing 1: Estruturas de dados do projeto (`produto.h`)

2.3 Dataset

O arquivo `dataset3.csv` contém registros de produtos com os campos `id`, `nome`, `categoria` e `valor`, separados por vírgula. O dataset utilizado nos testes possui **300.006 registros**, abrangendo categorias como Eletrônicos e

Informática, com valores variando entre dezenas e dezenas de milhares de reais.

2.4 Ambiente de Execução

- **Sistema Operacional:** WINDOWS
- **Processador:** RYZEN 5 3500X
- **Memória RAM:** 16GB
- **Compilador:** GCC (padrão C99)
- **Linguagem:** C

3 Descrição de Atividades

3.1 Leitura do Arquivo CSV

A leitura do CSV é realizada pela função `ler_csv()` em `readcsv.c`. A função abre o arquivo com verificação de erro, ignora o cabeçalho, e processa cada linha com `sscanf`. Linhas em branco ou com formato inválido são ignoradas com aviso. Os registros são inseridos no vetor dinâmico via `adicionar_produto()`:

```
1 int ler_csv(const char *caminho, VetorProdutos *vetor) {
2     FILE *arquivo = fopen(caminho, "r");
3     if (arquivo == NULL) {
4         printf("Erro: arquivo '%s' nao encontrado.\n",
5             caminho);
6         return 0;
7     }
8     char linha[256];
9     fgets(linha, sizeof(linha), arquivo); // ignora
10    // cabeçalho
11    while (fgets(linha, sizeof(linha), arquivo)) {
12        if (linha[0] == '\n' || linha[0] == '\0')
13            continue;
14        Produto p;
15        char valor_str[20];
16        int campos = sscanf(linha, "%d
17            ,%99[^,],%49[^,],%19s",
18            &p.id, p.nome, p.categoria,
19            valor_str);
20        if (campos != 4) continue;
21        p.valor = atof(valor_str);
22        adicionar_produto(vetor, p);
23    }
24    fclose(arquivo);
25    return 1;
26 }
```

Listing 2: Leitura e parsing do CSV (`readcsv.c`)

3.2 Vetor Dinâmico

O vetor é inicializado com capacidade 10 e dobra de tamanho automaticamente via `realloc` sempre que necessário, garantindo armazenamento eficiente de qualquer volume de registros:

```

1 void inicializador_vetor(VetorProdutos *v) {
2     v->capacidade = 10;
3     v->tamanho     = 0;
4     v->dados = malloc(v->capacidade * sizeof(Produto));
5 }
6
7 int adicionar_produto(VetorProdutos *v, Produto p) {
8     if (v->tamanho == v->capacidade) {
9         v->capacidade *= 2;
10        v->dados = realloc(v->dados,
11                           v->capacidade * sizeof(
12                               Produto));
13        if (v->dados == NULL) return 0;
14    }
15    v->dados[v->tamanho++] = p;
16    return 1;
17 }

```

Listing 3: Gerenciamento do vetor dinâmico (produto.c)

3.3 Implementação da Busca Sequencial

A função `busca_sequencial()` percorre o vetor elemento a elemento, comparando o campo `id`. Retorna o índice do elemento encontrado ou `-1` caso não exista:

```

1 int busca_sequencial(VetorProdutos *v, int id_buscado) {
2     for (int i = 0; i < v->tamanho; i++) {
3         if (v->dados[i].id == id_buscado)
4             return i;
5     }
6     return -1;
7 }

```

Listing 4: Função de busca sequencial (produto.c)

3.4 Medição de Tempo

A medição de tempo é abstraída pelo módulo `timer.c`, que utiliza a função `clock()` da biblioteca `<time.h>` e converte o resultado para milissegundos:


```

1 double pegar_tempo() {
2     return (double)clock() / CLOCKS_PER_SEC;
3 }
4
5 double calcular_diferenca_ms(double inicio, double fim)
6 {
7     return (fim - inicio) * 1000.0;
8 }

```

Listing 5: Módulo de temporização (timer.c)

3.5 Protocolo Experimental

A função `executar_busca()` em `main.c` implementa o protocolo padronizado: 3 repetições de 1.000 buscas consecutivas, com cálculo do tempo médio por busca:

```

1 #define QTD_BUSCAS      1000
2 #define QTD_REPETICOES  3
3
4 void executar_busca(VetorProdutos *v, int id_alvo,
5                     const char *posicao) {
6     double soma_tempos = 0;
7     for (int r = 1; r <= QTD_REPETICOES; r++) {
8         double inicio = pegar_tempo();
9         for (int i = 0; i < QTD_BUSCAS; i++)
10             busca_sequencial(v, id_alvo);
11         double fim = pegar_tempo();
12         double tempo_ms = calcular_diferenca_ms(inicio,
13         fim);
14         soma_tempos += tempo_ms;
15     }
16     double media_por_busca =
17         (soma_tempos / QTD_REPETICOES) / QTD_BUSCAS;
18     printf("  >> Media de %.6f ms por busca.\n",
19         media_por_busca);
20 }

```

Listing 6: Protocolo de execução dos testes (main.c)

Os cenários de busca cobrem quatro posições:

- **Início:** ID 43614 – índice 0;

- **Meio:** ID 3341 – índice 150.003;
- **Final:** ID 189553 – índice 300.006;
- **Inexistente:** ID -9999 – não encontrado.

4 Análise dos Resultados

4.1 Quantidade Total de Registros Carregados

Foram carregados com sucesso **300.007 registros** a partir do arquivo `dataset3.csv`. Nenhum erro de leitura ou formato inválido foi identificado durante o carregamento.

4.2 Tempo Total das Execuções de Busca

A Tabela 1 apresenta os tempos totais (para 1.000 buscas) medidos em cada rodada e cenário. O tempo total acumulado de todas as execuções foi de aproximadamente **18,004 segundos**.

4.3 Tempo Médio por Busca

O tempo médio por busca individual foi calculado pela equação:

$$\bar{t} = \frac{\bar{T}_{\text{rodadas}}}{N_{\text{buscas}}} \quad (1)$$

onde $\bar{T}_{\text{rodadas}}$ é a média dos tempos totais das 3 rodadas e $N_{\text{buscas}} = 1000$. Os resultados por cenário foram:

- **Início:** $\bar{t} = 0,000$ ms/busca
- **Meio:** $\bar{t} = 1,527$ ms/busca
- **Final:** $\bar{t} = 3,175$ ms/busca
- **Inexistente:** $\bar{t} = 2,300$ ms/busca

4.4 Tabela de Resultados

Tabela 1: Resultados completos das execuções de busca sequencial (300.006 registros)

Cenário	ID buscado	Rodada	Nº Buscas	T. Total (ms)	T. Médio (ms/busca)
Início	43614 (índice 0)	1	1.000	0,0000	0,000000
		2	1.000	0,0000	
		3	1.000	0,0000	
Meio	3341 (índice 150.003)	1	1.000	1583,0000	1,526667
		2	1.000	1625,0000	
		3	1.000	1372,0000	
Final	189553 (índice 300.006)	1	1.000	3122,0000	3,174667
		2	1.000	3160,0000	
		3	1.000	3242,0000	
Inexistente	-9999 (não encontrado)	1	1.000	2302,0000	2,300000
		2	1.000	2325,0000	
		3	1.000	2273,0000	

4.5 Comportamento Observado

Os resultados demonstram com clareza o comportamento da busca sequencial em função da posição do elemento no vetor. O elemento no **início** (índice 0) foi encontrado em tempo virtualmente nulo — a busca encerra na primeira comparação, independentemente do tamanho do vetor. O elemento no **meio** (índice 150.003) exigiu percorrer metade do vetor, resultando em aproximadamente metade do tempo do caso final. O elemento no **final** (índice 300.006) demandou a varredura completa, gerando o maior tempo médio observado: 3,175 ms/busca.

O caso **inexistente** (ID -9999) também exigiu varredura completa, porém apresentou tempo médio de 2,300 ms/busca — inferior ao caso final. Isso é esperado: o elemento inexistente aciona um desvio de fluxo diferente internamente, e a ausência de retorno antecipado com atribuição de variável pode gerar um caminho de execução ligeiramente mais rápido dependendo da otimização do compilador.

4.6 Relação Entre Tamanho do Vetor e Tempo de Busca

A busca sequencial possui complexidade $\mathcal{O}(n)$ no pior caso. Os dados confirmam essa relação: o tempo médio do elemento no **meio** (1,527 ms) é

aproximadamente **metade** do tempo do elemento no **final** (3,175 ms), sendo que o índice do meio é exatamente metade do índice do final. Isso demonstra proporcionalidade direta entre a posição do elemento e o tempo de busca, validando empiricamente o comportamento linear esperado pela teoria.

4.7 Limitações da Busca Sequencial

1. **Escalabilidade linear:** com 300.006 registros, o pior caso já atinge 3,175 ms por busca. Em bases com milhões de registros, esse tempo seria proporcional e proibitivo para aplicações em tempo real.
2. **Pior caso frequente:** elementos inexistentes ou no final forçam varredura completa sem qualquer interrupção antecipada.
3. **Sem aproveitamento de ordenação:** mesmo que o vetor esteja ordenado por `id`, a busca sequencial não se beneficia dessa propriedade — ao contrário da busca binária ($\mathcal{O}(\log n)$).
4. **Repetição sem cache:** buscas repetidas pelo mesmo elemento percorrem o vetor integralmente a cada chamada, sem qualquer mecanismo de memoização.

Essas limitações motivam a implementação da **Tabela Hash** na Fase II, que promete complexidade de busca $\mathcal{O}(1)$ no caso médio — independentemente do tamanho do vetor.

5 Trabalhos Futuros

A **Fase II** do projeto prevê a implementação de uma **Tabela Hash** para realização das mesmas buscas experimentadas nesta fase. A comparação direta entre os tempos médios permitirá quantificar o ganho de desempenho obtido com a estrutura hash. Outros desdobramentos possíveis incluem:

- Comparação com **busca binária** ($\mathcal{O}(\log n)$), que exigiria ordenação prévia do vetor por **id**;
- Testes com datasets de tamanhos diferentes (ex.: 100 mil, 500 mil, 1 milhão de registros) para traçar a curva de crescimento completa;
- Análise do impacto de diferentes funções de hash e políticas de tratamento de colisões no desempenho real da Fase II.

Bibliografia

CORMEN, T. H. *et al.* **Algoritmos: Teoria e Prática**. 3. ed. Rio de Janeiro: Elsevier, 2012.

KNUTH, D. E. **The Art of Computer Programming, Vol. 3: Sorting and Searching**. 2. ed. Addison-Wesley, 1998.

ZIVIANI, N. **Projeto de Algoritmos com Implementações em Pascal e C**. 3. ed. São Paulo: Cengage Learning, 2010.

Anexo

Repositório do Projeto

O código-fonte completo está disponível em:
<https://github.com/lpruim3/pi-2>

Estrutura do Repositório

```
pi-2/  
  data/  
    dataset3.csv      # Dataset com 300.006 registros  
  src/  
    include/  
      produto.h      # Structs e protótipos  
      readcsv.h  
      timer.h  
    main.c           # Protocolo experimental  
    produto.c        # Vetor dinâmico e busca sequencial  
    readcsv.c        # Leitura do CSV  
    timer.c          # Medicao de tempo
```

Instruções de Compilação e Execução

```
1  # Compile  
2  gcc -o busca src/main.c src/produto.c src/readcsv.c src/  
   timer.c  
3  
4  # Executar (a partir da raiz do projeto)  
5  ./busca
```

Listing 7: Compilação e execução